Electric Vehicle Data Analysis

Index

Section - 1

- **Data sets**

Column Name	Description
VIN (1-10)	First 10 characters of the Vehicle Identification Number (unique identifier for a vehicle)
County	The county where the vehicle is registered (e.g., King, Kitsap)
City	The city where the vehicle is registered (e.g., Seattle, Poulsbo)
State	The state of registration (WA - Washington)
Postal Code	The ZIP code where the vehicle is registered
Model Year	The year the vehicle was manufactured (e.g., 2019, 2020, 2023)
Make	The brand of the vehicle (e.g., Tesla, Hyundai)
Model	The specific model of the vehicle (e.g., Model 3, Model Y, Ioniq 5)
Electric Vehicle Type	The type of EV (Battery Electric Vehicle - BEV)
Clean Alternative Fuel Vehicle (CAFV) Eligibility	Whether the vehicle qualifies for clean energy incentives
Electric Range	The estimated range of the vehicle in miles on a full charge
Base MSRP	Manufacturer's Suggested Retail Price (MSRP) for the base model (0 means data is missing)
Legislative District	The legislative district where the vehicle is registered
DOL Vehicle ID	Unique ID assigned by the Department of Licensing (Washington)
Vehicle Location	GPS coordinates (longitude, latitude) of the vehicle's registered location
Electric Utility	The energy provider for the area (e.g., Puget Sound Energy, City of Seattle)
2020 Census Tract	The Census tract for demographic and geographic analysis

Data Clearing

1. VIN (1–10): First 10 characters of the Vehicle Identification Number

What it means

- **A VIN is a 17-character alphanumeric code unique to each vehicle.**
- **The dataset keeps only the first 10 characters to:**
  - **Identify the vehicle without exposing the full VIN**
  - **Preserve privacy while still allowing grouping/analysis**

- **Full VIN (Origin)-:**

  5YJ3E1EA7JF012345

- **VIN (1–10) stored in dataset-:**

  5YJ3E1EA7J

- **Python (Pandas) Code to Generate VIN (1–10):-**

```
df["VIN (1-10)"] = df["VIN"].str[:10]
```

Output:

VIN (full)	VIN (1-10)
5YJ3E1EA7JF012345	5YJ3E1EA7J
1HGCM82633A004352	1HGCM82633
JN1AZ0CP9CT012789	JN1AZ0CP9C

- **SQL Version:**

```
SELECT
  SUBSTRING(VIN, 1, 10) AS "VIN
(1-10)"
FROM vehicles;
```

Section- 2

Data Explorations

2. County – County where the vehicle is registered (e.g., India, South Africa)

- **Python (Pandas) Code**

Selecting the *County* column-:

```
import pandas as pd
data = {
   'Vehicle_ID': [101, 102, 103],
   'County': ['India', 'South Africa', 'China'],
   'Registration_Year': [2020, 2021, 2019]
}

df = pd.DataFrame(data)
county_column = df['County']

print(county_column)
```

Alternative ways to select the column

Using double brackets (returns a DataFrame):

- **Using .loc:**

```
county_df = df[['County']]
county_column = df.loc[:, 'County']
```

3. The city where the vehicle is registered (e.g., Pune , Bhusawal)

- **Selecting the City column**

```
import pandas as pd
    'Vehicle_ID': [201, 202, 203],
    'City': ['Bhusawal', 'Pune', 'Mumbai'],
    'County': ['India', 'India', 'India']
}

df = pd.DataFrame(data)

# Select the City column
city_column = df['City']

print(city_column)
```

Other common options
As a DataFrame:

```
city_df = df[['City']]
```

Using .loc:

city_column = df.loc[:, 'City']

pune_vehicles = df[df['City'] == 'Pune']

Section- 3

Data Visualization

4. The state of registration (WA - Washington)

Validate & Map Maharashtra State Code using Pandas

```
import pandas as pd
df = pd.DataFrame({
    "state_of_registration": [
        "Maha-Maharashtra",
        "Maharashtra",
        "MH",
        "Gujarat"
    ]
})
```

```
# State code mapping
state_code_map = {
    "maha-maharashtra": 27,
    "maharashtra": 27,
    "mh": 27,
    "gujarat": 24
}


# Normalize text and map code
df["state_code"] = (
    df["state_of_registration"]
    .str.lower()
    .str.strip()
    .map(state_code_map)
)
print(df)
```

Output

	state_of_registration	state_code
0	Maha-Maharashtra	27
1	Maharashtra	27
2	MH	27
3	Gujarat	24

5. The ZIP code where the vehicle is registered

ZIP Code Check (Python)

```
import re

def check_zip_code(zip_code):
    pattern = r"^\d{5}(-\d{4})?$"
    if re.match(pattern, zip_code):
        return "Valid ZIP code"
    else:
        return "Invalid ZIP code"
zip_codes = ["90210", "12345-6789", "ABCDE", "1234"]

for z in zip_codes:
    print(f"{z}: {check_zip_code(z)}")
```

Output

```
90210: Valid ZIP code
12345-6789: Valid ZIP code
ABCDE: Invalid ZIP code
1234: Invalid ZIP code
```

6. The year the vehicle was manufactured (e.g., 2023, 2024, 2025)

```
import pandas as pd

# Sample data
data = {
    'vehicle_id': [101, 102, 103, 104],
    'manufacture_year': [2023, 2024, 2025, 2023]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display the DataFrame
print("Original DataFrame:")
print(df)
```

```
# Filter rows with valid years (e.g., 2023, 2024, 2025)
valid_years = [2023, 2024, 2025]
df_valid = df[df['manufacture_year'].isin(valid_years)]

print("\nFiltered DataFrame (valid years 2023-2025):")
print(df_valid)
```

Output:

Original DataFrame:

	vehicle_id	manufacture_year
0	101	2023
1	102	2024
2	103	2025
3	104	2023

Filtered DataFrame (valid years 2023-2025):

	vehicle_id	manufacture_year
0	101	2023
1	102	2024
2	103	2025
3	104	2023

7. The brand of the vehicle (e.g., Tesla, Hyundai)

```
import pandas as pd

# Sample dataset
data = {
    "Vehicle": ["Model S", "Ioniq 5", "Civic", "Mustang", "Tucson"],
    "Brand": ["Tesla", "Hyundai", "Honda", "Ford", "Hyundai"],
    "Year": [2022, 2021, 2020, 2019, 2023]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display DataFrame
print("Original DataFrame:")
print(df)

# Check the unique brands
unique_brands = df['Brand'].unique()
print("\nUnique vehicle brands:")
print(unique_brands)
```

```
# Count of vehicles by brand
brand_counts = df['Brand'].value_counts()
print("\nCount of vehicles by brand:")
print(brand_counts)
```

Output:

```
Original DataFrame:
   Vehicle    Brand  Year
0   Model S    Tesla  2022
1  Ioniq 5  Hyundai  2021
2     Civic    Honda  2020
3   Mustang     Ford  2019
4    Tucson  Hyundai  2023

Unique vehicle brands:
['Tesla' 'Hyundai' 'Honda' 'Ford']

Count of vehicles by brand:
Hyundai    2
Tesla      1
Honda      1
```

Ford 1

Name: Brand, dtype: int64

Section- 4

Linear Regression Model

8. The specific model of the vehicle (e.g., Model 3, Model Y, Ioniq 5

```
import pandas as pd
# Example data
data = {
    'vehicle_name': [
        'Tesla Model 3 2021',
        'Tesla Model Y Long Range',
        'Hyundai Ioniq 5 EV',
        'Ford Mustang Mach-E',
        'BMW iX3'
    ]
}
# Create DataFrame
df = pd.DataFrame(data)


# Function to extract the model (everything after the brand name)
def extract_model(name):
```

```
    # Split by space and remove the brand (first word)
    parts = name.split(' ')
    if len(parts) > 1:
        # Return all except first word (the brand)
        return ' '.join(parts[1:])
    else:
        return name

# Apply the function
df['model'] = df['vehicle_name'].apply(extract_model)

# Show result
print(df)
```

output:

```
          vehicle_name              model
0       Tesla Model 3 2021        Model 3 2021
1  Tesla Model Y Long Range  Model Y Long Range
2       Hyundai Ioniq 5 EV       Ioniq 5 EV
3     Ford Mustang Mach-E     Mustang Mach-E
4              BMW iX3              iX3
```

9. The type of EV (Battery Electric Vehicle - BEV)

```
import pandas as pd

# Sample dataset of vehicles
data = {
    'Vehicle': ['Tesla Model 3', 'Toyota Corolla', 'Nissan Leaf', 'Ford F-150', 'Chevy Bolt'],
    'Type': ['BEV', 'Gasoline', 'BEV', 'Gasoline', 'BEV'],
    'Price ($)': [45000, 20000, 30000, 40000, 36000]
}

# Create DataFrame
df = pd.DataFrame(data)

# Filter only Battery Electric Vehicles (BEV)
bev_df = df[df['Type'] == 'BEV']
print(bev_df)
```

Output :

```
         Vehicle Type  Price ($)
0  Tesla Model 3  BEV      45000
2     Nissan Leaf  BEV      30000
4       Chevy Bolt  BEV      36000
```

10. Whether the vehicle qualifies for clean energy incentives

```
import pandas as pd

data = [
    {"make": "Tesla", "model": "Model Y", "year": 2024, "msrp": 52000, "battery_kwh": 75, "final_assembly_us": True},
    {"make": "Ford", "model": "F-150 Lightning", "year": 2023, "msrp": 82000, "battery_kwh": 98, "final_assembly_us": True},
    {"make": "Hyundai", "model": "Ioniq 5", "year": 2024, "msrp": 48000, "battery_kwh": 77, "final_assembly_us": False},
]

df = pd.DataFrame(data)

Eligibility Rules (Simplified US 30D)
def clean_vehicle_eligible(row):
    if row["year"] < 2023:
        return False
    if row["battery_kwh"] < 7:
        return False
    if not row["final_assembly_us"]:
```

```
        return False
    if row["msrp"] > 80000:
        return False
    return True


df["clean_energy_eligible"] = df.apply(clean_vehicle_eligible, axis=1)
df
```

output:

	make	model	year	msrp	battery_kwh	final_assembly_us	clean_energy_eligible
0	Tesla	Model Y	2024	52000	75	True	True
1	Ford	F-150 Lightning	2023	82000	98	True	False
2	Hyundai	Ioniq 5	2024	48000	77	False	False

11. The estimated range of the vehicle in miles on a full charge

```
import pandas as pd


# Sample vehicle data
data = {
    "Vehicle": ["EV A", "EV B", "EV C"],
    "Battery_kWh": [60, 75, 100],
    "Efficiency_mi_per_kWh": [4.0, 3.5, 3.0]
}


df = pd.DataFrame(data)


# Calculate estimated range
df["Estimated_Range_miles"] = df["Battery_kWh"] *
df["Efficiency_mi_per_kWh"]


print(df)
```

Output

```
  Vehicle  Battery_kWh  Efficiency_mi_per_kWh
Estimated_Range_miles
0    EV A          60                   4.0                 240.0
1    EV B          75                   3.5                 262.5
```

2 EV C 100 3.0 300.0

Manufacturer's Suggested Retail Price (MSRP) for the base model (0 means data is missing)

```
import pandas as pd

import numpy as np


# Replace 0 MSRP with NaN (mark as missing)

df['MSRP'] = df['MSRP'].replace(0, np.nan)


# Filter base model only

base_model_msrp = df.loc[df['Trim'] == 'Base', ['Make',
'Model', 'Year', 'MSRP']]


# Display result

print(base_model_msrp)
```

output:

	Make	Model	Year	MSRP
0	Toyota	Corolla	2023	21900.0
3	Honda	Civic	2023	NaN
7	Ford	Focus	2022	20850.0

Optional: count missing MSRP values for base models

```
missing_count = base_model_msrp['MSRP'].isna().sum()
print("Missing MSRP (Base model):", missing_count)
```

12. The legislative district where the vehicle is registered
check code pandas with output

```
import pandas as pd

data = {
   "vehicle_id": [101, 102, 103, 104],
   "owner_name": ["Alice", "Bob", "Charlie", "Diana"],
   "legislative_district": ["District 5", "District 3", "District 5", "District 7"]
}

df = pd.DataFrame(data)
df
```

Output:

```
   vehicle_id owner_name legislative_district
0         101      Alice          District 5
1         102        Bob          District 3
2         103    Charlie          District 5
3         104      Diana          District 7
```

Check Vehicles Registered in a Specific Legislative District

```
district_to_check = "District 5"

result = df[df["legislative_district"] == district_to_check]
print(result)
```

output:

```
   vehicle_id owner_name legislative_district
0         101      Alice          District 5
2         103    Charlie          District 5
```

Count Vehicles per Legislative District

```
district_counts = df["legislative_district"].value_counts()
print(district_counts)
```

Output:

```
District 5    2
District 3    1
District 7    1
Name: legislative_district, dtype: int64
```

Check if a Vehicle Is Registered in a Given District

```
vehicle_id = 102
district = "District 5"

check = not df[
   (df["vehicle_id"] == vehicle_id) &
   (df["legislative_district"] == district)
].empty

print(check)
```

Output:

Fa

13. Unique ID assigned by the Department of Licensing (Washington)

```
import pandas as pd


df = pd.DataFrame({"wa_id": ["WDL12345678", "WDL87654321"]})
df["is_valid"] = df["wa_id"].apply(validate_wa_dol_id)
print(df)
```

14. GPS coordinates (longitude, latitude) of the vehicle's registered location

```
import pandas as pd


# Sample vehicle registration data
data = {
    "vehicle_id": ["V001", "V002", "V003", "V004"],
    "longitude": [77.5946, -181.0, 120.9822, 85.3240],
    "latitude": [12.9716, 45.0, -91.5, 27.7172]
}
```

```
df = pd.DataFrame(data)


# Function to check GPS validity
def is_valid_gps(lon, lat):
    return (-180 <= lon <= 180) and (-90 <= lat
<= 90)


# Apply validation
df["gps_valid"] = df.apply(
    lambda row: is_valid_gps(row["longitude"],
row["latitude"]),
    axis=1
)


print(df)
```

Output

	vehicle_id	longitude	latitude	gps_valid
0	V001	77.5946	12.9716	True
1	V002	-181.0000	45.0000	False
2	V003	120.9822	-91.5000	False
3	V004	85.3240	27.7172	True

15. The energy provider for the area (e.g., Puget Sound Energy, City of Seattle)

```
import pandas as pd
# Input data (areas to check)
df = pd.DataFrame({
   "city": ["Seattle", "Bellevue", "Tacoma", "Spokane"]
})

# Energy provider lookup table
providers = pd.DataFrame({
   "city": ["Seattle", "Bellevue", "Tacoma", "Spokane"],
   "energy_provider": [
      "Seattle City Light",
      "Puget Sound Energy",
      "Tacoma Power",
      "Avista Utilities"
   ]
})

# Merge to assign energy provider
result = df.merge(providers, on="city", how="left")

print(result)
```

Output:

	city	energy_provider
0	Seattle	Seattle City Light
1	Bellevue	Puget Sound Energy
2	Tacoma	Tacoma Power
3	Spokane	Avista Utilities

Alternative: Using a Dictionary (Simpler)

```
provider_map = {
    "Seattle": "Seattle City Light",
    "Bellevue": "Puget Sound Energy",
    "Tacoma": "Tacoma Power",
    "Spokane": "Avista Utilities"
}

df["energy_provider"] = df["city"].map(provider_map)

print(df)
```

16. The Census tract for demographic and geographic analysis

```
import pandas as pd


data = {
   "GEOID": ["06075010100", "36061000100", "12086000101", "06073008302", "INVALID123"],
   "population": [4500, 3800, 5200, 6100, 3000],
   "median_income": [72000, 65000, 54000, 81000, 40000]
}


df = pd.DataFrame(data)
df
```

output:

	GEOID	population	median_income
0	06075010100	4500	72000
1	36061000100	3800	65000
2	12086000101	5200	54000
3	06073008302	6100	81000
4	INVALID123	3000	40000

Census Tract Validation Check

```
# Ensure GEOID is string
df["GEOID"] = df["GEOID"].astype(str)

# Check if GEOID is valid Census Tract (11 digits)
df["valid_tract"] = df["GEOID"].str.match(r"^\d{11}$")

df
```

output:

	GEOID	population	median_income	valid_tract
0	06075010100	4500	72000	True
1	36061000100	3800	65000	True
2	12086000101	5200	54000	True
3	06073008302	6100	81000	True
4	INVALID123	3000	40000	False

Extract Geographic Components

```
df["state_fips"] = df["GEOID"].str[:2]
df["county_fips"] = df["GEOID"].str[2:5]
df["tract_code"] = df["GEOID"].str[5:]
```

```
df
```

output:

	GEOID	population	median_income	valid_tract	state_fips	county_fips	tract_code
0	06075010100	4500	72000	True	06	075	010100
1	36061000100	3800	65000	True	36	061	000100
2	12086000101	5200	54000	True	12	086	000101
3	06073008302	6100	81000	True	06	073	008302
4	INVALID123	3000	40000	False	IN	VAL	ID123

Basic Demographic Analysis (Valid Tracts Only)

```
analysis = (
    df[df["valid_tract"]]
    .groupby("state_fips")
    .agg(
        total_population=("population", "sum"),
        avg_income=("median_income", "mean"),
        tract_count=("GEOID", "count")
    )
)
```

Analysis

Output:

state_fips	total_population	avg_income	tract_count
06	10600	76500.0	2
12	5200	54000.0	1
36	3800	65000.0	1